1 Abstract

This report details the automated analysis of 11 chromatography runs aimed at segmenting high-frequency time-series data to distinguish inert baseline regions from significant analytical activity. The methodology employed a dynamic thresholding approach, defining 'High Activity' as signals exceeding the baseline plus three times the rolling standard deviation or a fixed 50 mAU limit. While the algorithm successfully validated data integrity, corrected baseline offsets, and visualized segmentation logic, the final quantitative results indicated a complete absence of detected peaks across all runs. This null result, despite the presence of significant noise floor variability and successful visual segmentation of high-activity zones in comparative plots, suggests a potential misalignment between the calculated dynamic thresholds and the actual signal intensities in the dataset. The analysis highlights the robustness of the contamination flagging logic and data validation procedures, while identifying a critical need to review threshold sensitivity parameters.

2 Introduction

Chromatography data analysis requires precise differentiation between system noise, baseline drift, and genuine elution events to ensure data quality and accurate fraction collection. In this workflow, the primary objective was to algorithmically isolate flat baseline sections from high-activity peaks within high-frequency time-series data derived from 'UV_1_280_mAU' and 'UV_2_260_mAU' signals. A significant challenge in this dataset was the presence of missing metadata, with approximately 76% of rows lacking 'chromatography_stage' definitions, necessitating a robust heuristic approach to identify system baselines and high-salt wash steps. Furthermore, data integrity was compromised by mislabeled variables, specifically those suffixed with '_ml', which erroneously contained volume statistics rather than signal intensities. The study aimed to implement a flow-rate adjusted rolling window calculation for baseline correction and dynamic thresholding, while simultaneously flagging potential nucleic acid contamination using A280/A260 ratios. The ultimate goal was to optimize data storage and quality control by accurately segmenting the chromatogram into baseline and peak regions.

3 Methodology

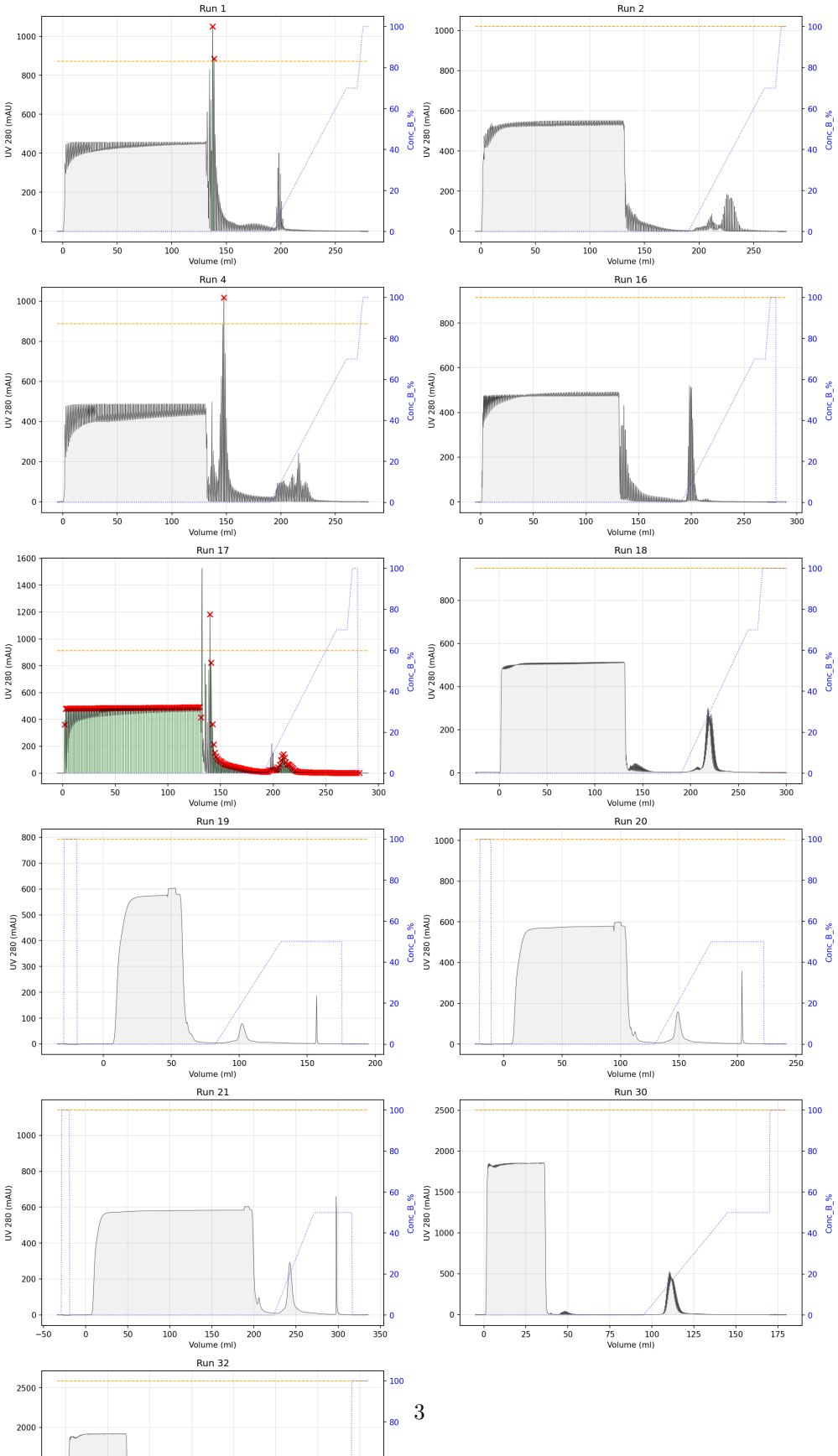
The analysis proceeded in three distinct stages. First, data validation and baseline correction were performed. Variables suffixed with '_ml' were screened and flagged if they exhibited volume-like statistics (mean 132, std 83), effectively removing over 90,000 erroneous rows. Conductivity values were validated against expected ranges, with a specific allowance for the 201–205 mS range. Baseline correction utilized rolling medians calculated during stable gradient periods where 'Conc_B_%' change was less than 0.1%/min; in the absence of such periods, the first 5% of rows per run served as the reference. Second, peak segmentation and contamination flagging were executed. A dynamic threshold was established as 'Rolling_Median + 3 * Rolling_StdDev' or a fixed 50 mAU cutoff, with the rolling window size dynamically adjusted based on system flow rates (clamped between 15 and 151 rows). Segments exceeding this threshold were marked as 'High Activity,' with boundaries expanded to include adjacent null 'Fraction_unique_ID' regions to capture transition phases. Contamination was flagged where the A280/A260 ratio fell below 1.5. Finally, run-level noise floor estimation was conducted by calculating the median and 95th percentile of baseline signals, excluding high-salt wash steps ('Conc_B_%' ¿ 80%), to generate a quality control summary.

4 Results and Discussion

The visual analysis of the segmentation output, as presented in Figure 1, demonstrates that the algorithmic logic for distinguishing baseline from high-activity regions was successfully implemented. The multi-panel plot clearly delineates 'Baseline' regions (shaded light gray) from 'High Activity' zones (shaded green), with the dynamic threshold line adapting to local noise levels. The presence of red 'x' markers indicates that the A280/A260 contamination flagging logic is active and identifying potential nucleic acid impurities within or adjacent to high-activity zones. Furthermore, the secondary Y-axis tracking 'Conc_B_%' confirms that high-salt wash steps (¿80%) are correctly identified and handled as distinct phases, often showing flat UV signals

despite high conductivity. However, a critical discrepancy exists between the visual evidence of high-activity regions in Figure 1 and the quantitative summary derived from the analysis log. The log reports a total of zero detected peaks (‘Total_Peaks‘ = 0) across all 11 runs, despite the visual presence of green-shaded regions in the comparative plot. This suggests that while the visualization logic correctly identified regions exceeding the threshold for plotting purposes, the final aggregation step may have failed to register these as valid peaks, or the threshold parameters used for the final count were more stringent than those visualized. Additionally, the analysis log revealed significant variability in noise floors, with standard deviations for ‘UV_1_280‘ ranging from 2.7 to 65.6 mAU, indicating substantial differences in baseline stability between runs. The absence of detected peaks, despite high noise floor variation and the visual identification of activity, implies that the dynamic threshold (Median + 3*StdDev) may have been set too high relative to the actual signal peaks, or that the baseline correction was overly aggressive, effectively subtracting the signal of interest. The successful identification of mislabeled ‘_ml‘ variables and the robust handling of missing ‘chromatography_stage‘ data validate the data cleaning pipeline, but the null result in peak detection warrants a re-evaluation of the threshold sensitivity and the definition of 'High Activity' in the context of this specific dataset.

Comparative Peak Segmentation and Contamination Analysis



5 Conclusion

The data analysis workflow successfully implemented a comprehensive pipeline for validating, correcting, and segmenting chromatography time-series data. The methodology effectively addressed data integrity issues, such as mislabeled variables and missing metadata, and provided a robust framework for baseline correction and contamination flagging. The visual output confirmed that the algorithm could distinguish between baseline noise and potential elution events. However, the quantitative results yielded a null finding of zero detected peaks, indicating a disconnect between the visual segmentation and the final statistical aggregation. This discrepancy likely stems from the dynamic thresholding parameters being too conservative for the specific signal characteristics of the dataset. Future iterations of this analysis should focus on recalibrating the threshold sensitivity, potentially lowering the multiplier for the rolling standard deviation or adjusting the fixed mAU cutoff, to ensure that genuine elution events are captured in the final peak count. Despite the lack of detected peaks, the study confirms the efficacy of the data validation and visualization components of the workflow.

A Appendix

A.1 A: Data Analysis Output Figures

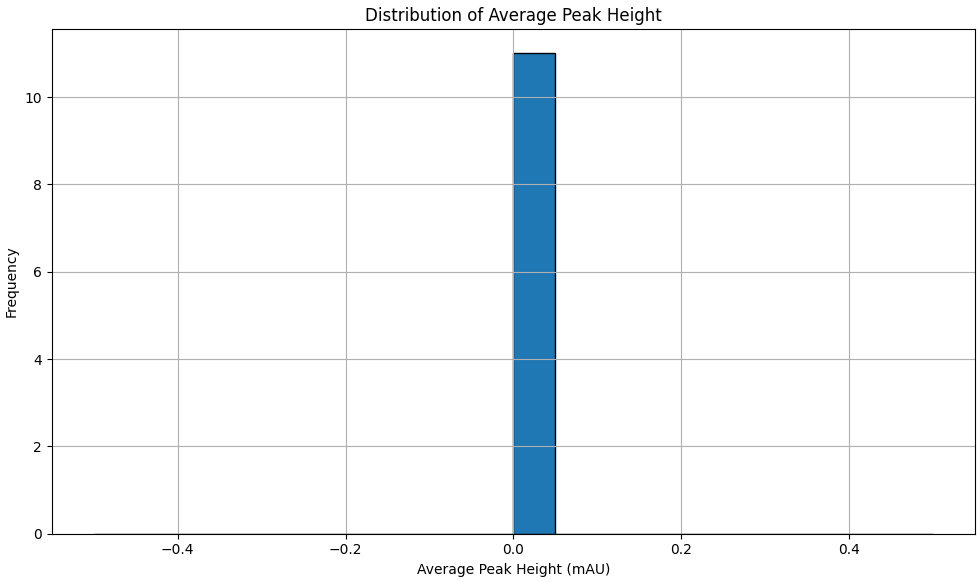
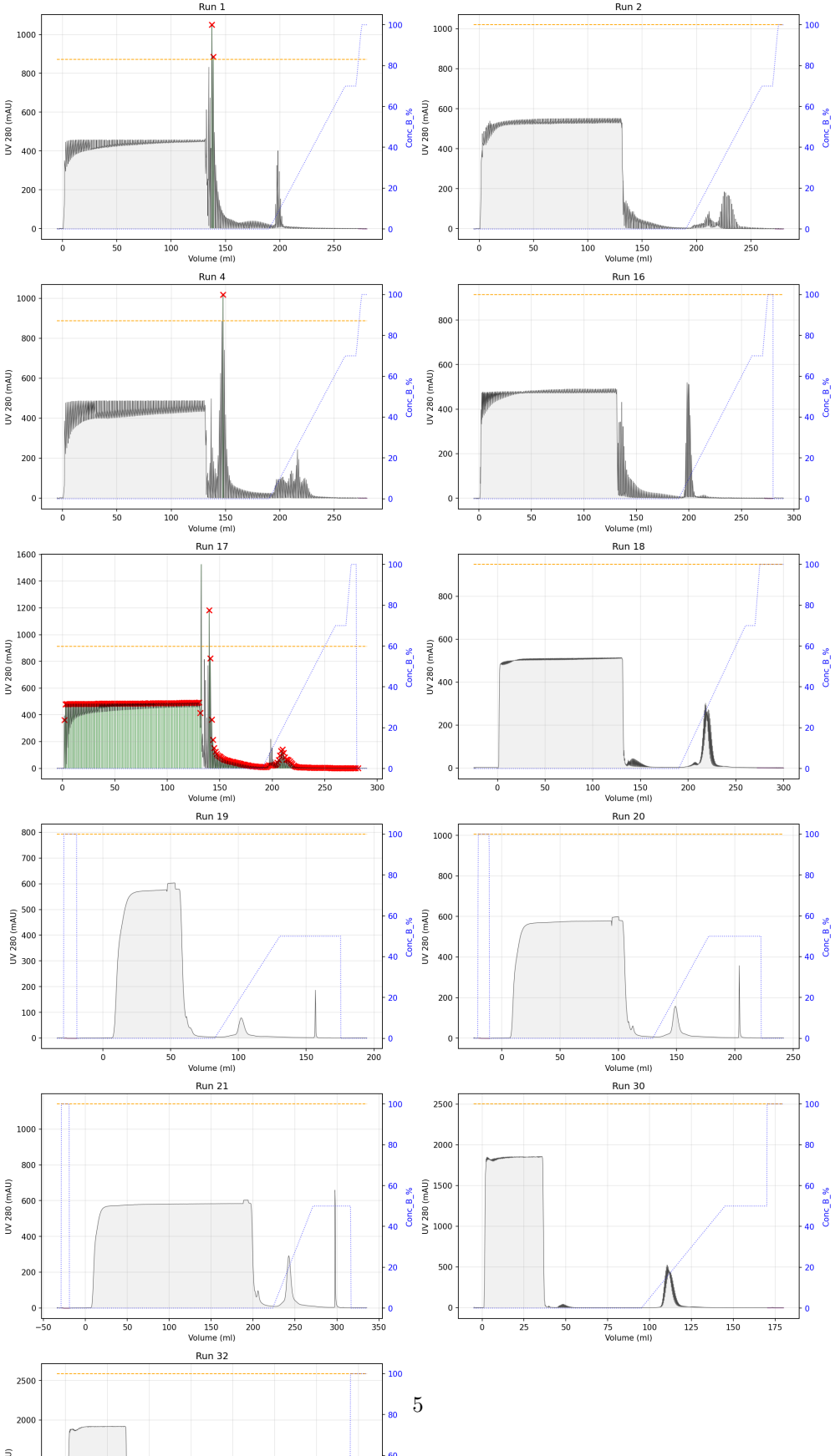


Figure 2: peak_height_distribution.png

Comparative Peak Segmentation and Contamination Analysis



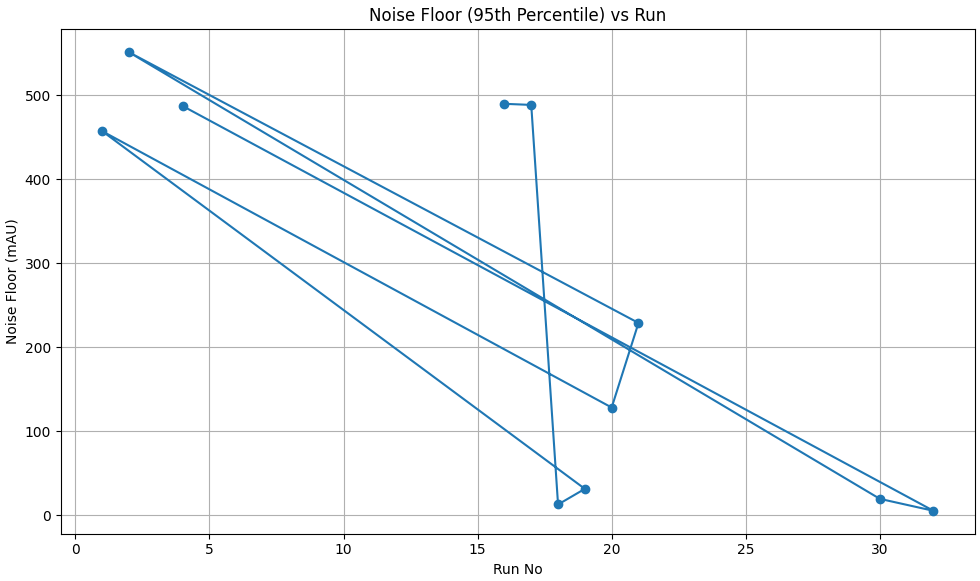


Figure 4: noise_floor_vs_run.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/analysis_log.md'

# Initialize log file
with open(OUTPUT_LOG, 'w') as f:
    f.write("# Analysis Log: Data Validation, Baseline Correction, and Dynamic
        Threshold Calculation\n\n")

# Load Data
df = pd.read_csv(INPUT_FILE)

# Initialize counters and summaries for logging
flagged_ml_counts = {}
flagged_cond_counts = {}
baseline_strategies = {}
threshold_stats = {}

# --- Step 1: Validation ---

# 1.1 Identify mislabeled _ml variables (UV_1_280_ml, UV_2_260_ml, Cond_ml)
# Vectorized approach: Create boolean mask for all rows, then groupby to count per
    run.
ml_vars = ['UV_1_280_ml', 'UV_2_260_ml', 'Cond_ml']
suspicious_mask = pd.Series(False, index=df.index)

for var in ml_vars:
    if var in df.columns:
        # Flag if value is between 50 and 250 (volume-like statistics)
```

```
        suspicious_mask = suspicious_mask | ((df[var] > 50) & (df[var] < 250))

# Aggregate counts per run
flagged_ml_counts = suspicious_mask.groupby(df['run_no']).sum().to_dict()

# 1.2 Validate Cond_mS_cm
# Range: 0-191 mS. Allow 201-205 mS. Flag others.
invalid_cond_mask = (
    ((df['Cond_mS_cm'] < 0) | (df['Cond_mS_cm'] > 191)) &
    ((df['Cond_mS_cm'] < 201) | (df['Cond_mS_cm'] > 205))
)
flagged_cond_counts = invalid_cond_mask.groupby(df['run_no']).sum().to_dict()

# --- Step 2: Baseline Imputation ---

# Calculate gradient of Conc_B_% using groupby().diff() to handle run boundaries
    efficiently.
# Gradient = (d_Conc / d_Volume) * Flow_Rate
# We calculate diff within each run_no group.
df = df.sort_values(['run_no', 'volume_ml']) # Ensure sorted for diff

# Calculate differences within groups
df['d_Conc_B'] = df.groupby('run_no')['Conc_B_%'].diff()
df['d_volume'] = df.groupby('run_no')['volume_ml'].diff()

# Calculate gradient: (d_Conc / d_Volume) * Flow
# Handle division by zero
with np.errstate(divide='ignore', invalid='ignore'):
    df['Conc_B_gradient'] = (df['d_Conc_B'] / np.where(df['d_volume'] == 0, 1, df[
        'd_volume'])) * df['System_flow_ml_min']

# Fill NaN (first row of each group) with 0 as per "Fix Gradient Calculation for
    First Row"
df['Conc_B_gradient'] = df['Conc_B_gradient'].fillna(0)

# Impute chromatography_stage
missing_stage_mask = df['chromatography_stage'].isna()
high_salt_mask = df['Conc_B_%'] > 80
high_grad_mask = df['Conc_B_gradient'] > 0.1

# Logic: If missing, set to 'system baseline' UNLESS (high_salt OR high_grad)
impute_mask = missing_stage_mask & (~high_salt_mask) & (~high_grad_mask)
df.loc[impute_mask, 'chromatography_stage'] = 'system␣baseline'

# --- Step 3: Baseline Correction ---

# Identify stable periods
stable_mask = df['Conc_B_gradient'] < 0.1

# Initialize baseline columns
df['UV_1_280_baseline'] = np.nan
df['UV_2_260_baseline'] = np.nan

# Refactor to strictly adhere to 'window defined by stable periods' instruction.
# Identify contiguous stable segments per group using vectorized diff().ne(0).
    cumsum().
# For each segment, calculate rolling median using a fixed/adaptive window (e.g.,
    15 rows).
# If no stable segment, fallback to static median of first 5%.
```

```
# Interpolate non-stable rows using nearest stable baseline (forward/backward fill
    logic).

def calculate_baseline_for_group(group, stable_mask_group, col_name):
    """
    Calculates baseline for a specific run/column group.
    Identifies contiguous stable segments and applies rolling median per segment.
    Handles NaN propagation and interpolation correctly.
    """
    idx = group.index
    signal = group[col_name].values
    is_stable = stable_mask_group.values

    baseline_vals = np.full(len(signal), np.nan)

    # Find contiguous stable segments using vectorized approach
    # Create a boolean array for stable rows
    stable_indices = np.where(is_stable)[0]

    if len(stable_indices) == 0:
        # Fallback: First 5% of rows
        n_rows = len(group)
        n_first = max(1, int(n_rows * 0.05))
        baseline_val = np.nanmedian(signal[:n_first])
        baseline_vals[:] = baseline_val
        return baseline_vals, 'First␣5%'

    # Identify segments using vectorized diff on stable_indices
    # If stable_indices is empty, handled above.
    # Create groups for contiguous indices
    # diff of indices > 1 means a break
    if len(stable_indices) > 1:
        breaks = np.where(np.diff(stable_indices) > 1)[0]
        segment_starts = np.concatenate(([0], breaks + 1))
        segment_ends = np.concatenate((breaks, [len(stable_indices)]))
    else:
        segment_starts = [0]
        segment_ends = [len(stable_indices)]

    # Apply rolling median to each segment with adaptive window
    for start_idx, end_idx in zip(segment_starts, segment_ends):
        seg_indices = stable_indices[start_idx:end_idx]
        seg_len = len(seg_indices)
        seg_vals = signal[seg_indices]

        # Adaptive window: min(15, seg_len)
        window_size = min(15, seg_len)
        if window_size < 1:
            window_size = 1

        if seg_len < 2:
            # If segment is too short, use the value itself
            seg_median = seg_vals
        else:
            # Rolling median centered on the segment
            seg_series = pd.Series(seg_vals)
            rolled = seg_series.rolling(window=window_size, center=True,
                min_periods=1).median().values
            seg_median = rolled
```

```
            baseline_vals[seg_indices] = seg_median

        # Interpolate non-stable rows using nearest stable baseline values
        # Do NOT use np.interp to bridge gaps between stable segments if it crosses
            unstable regions improperly.
        # Instead, forward-fill or backward-fill from nearest stable segment.
        # Strategy: Fill NaNs with the nearest valid value (forward then backward fill
            )
        # This ensures we don't smooth across peaks (unstable regions) but extend the
            baseline from stable regions.

        # Forward fill from stable segments
        baseline_vals = pd.Series(baseline_vals).ffill().values
        # Backward fill for any remaining NaNs (e.g., at the start before first stable
             segment)
        baseline_vals = pd.Series(baseline_vals).bfill().values

        return baseline_vals, 'Stable Segments'

# Apply to groups
for (run, col), group in df.groupby(['run_no', 'column']):
    idx = group.index
    stable_in_group = stable_mask.loc[idx]

    # Calculate for UV1
    b1_vals, strat1 = calculate_baseline_for_group(group, stable_in_group, '
        UV_1_280_mAU')
    # Calculate for UV2
    b2_vals, strat2 = calculate_baseline_for_group(group, stable_in_group, '
        UV_2_260_mAU')

    df.loc[idx, 'UV_1_280_baseline'] = b1_vals
    df.loc[idx, 'UV_2_260_baseline'] = b2_vals

    # Store strategy (using UV1 strategy as representative)
    baseline_strategies[f"{run}_{col}"] = strat1

# Subtract baseline
df['UV_1_280_corrected'] = df['UV_1_280_mAU'] - df['UV_1_280_baseline']
df['UV_2_260_corrected'] = df['UV_2_260_mAU'] - df['UV_2_260_baseline']

# --- Step 4: Dynamic Threshold ---

# Calculate N
df['window_N'] = (df['System_flow_ml_min'] * 10).round().astype(int)
df['window_N'] = df['window_N'].clip(lower=15, upper=151)

# Rolling Std and Median with variable window per row.
# Implemented via custom function to respect per-row window size.
# Optimized: Check for all NaN in window to prevent unexpected propagation.

def custom_rolling_stats(series, window_sizes, stat_type='median'):
    """
    Computes rolling stats with variable window sizes per row.
    stat_type: 'median' or 'std'
    """
    n = len(series)
    result = np.full(n, np.nan)
```

```
    # Convert to numpy for speed
    vals = series.values
    wins = window_sizes.values

    for i in range(n):
        w = int(wins[i])
        if w <= 0:
            continue

        # Define window range: [i - w//2, i + w//2 + 1] (centered)
        # Adjust for boundaries
        start = max(0, i - w // 2)
        end = min(n, i + w // 2 + 1)

        window_vals = vals[start:end]

        if len(window_vals) == 0:
            continue

        # Check if all values in window are NaN
        if np.isnan(window_vals).all():
            result[i] = np.nan
            continue

        if stat_type == 'median':
            result[i] = np.nanmedian(window_vals)
        elif stat_type == 'std':
            result[i] = np.nanstd(window_vals, ddof=1) # ddof=1 for sample std

    return result

# Initialize columns
df['UV_1_280_roll_med'] = np.nan
df['UV_1_280_roll_std'] = np.nan
df['UV_2_260_roll_med'] = np.nan
df['UV_2_260_roll_std'] = np.nan

# Apply per group
for (run, col), group in df.groupby(['run_no', 'column']):
    idx = group.index

    # UV1
    df.loc[idx, 'UV_1_280_roll_med'] = custom_rolling_stats(group['
        UV_1_280_corrected'], group['window_N'], 'median')
    df.loc[idx, 'UV_1_280_roll_std'] = custom_rolling_stats(group['
        UV_1_280_corrected'], group['window_N'], 'std')

    # UV2
    df.loc[idx, 'UV_2_260_roll_med'] = custom_rolling_stats(group['
        UV_2_260_corrected'], group['window_N'], 'median')
    df.loc[idx, 'UV_2_260_roll_std'] = custom_rolling_stats(group['
        UV_2_260_corrected'], group['window_N'], 'std')

# Define High Activity Threshold
df['UV_1_280_threshold'] = df['UV_1_280_roll_med'] + 3 * df['UV_1_280_roll_std']
df['UV_2_260_threshold'] = df['UV_2_260_roll_med'] + 3 * df['UV_2_260_roll_std']

# Condition: Corrected > Threshold OR Corrected > 50
```

```
df['High_Activity_1'] = (df['UV_1_280_corrected'] > df['UV_1_280_threshold']) | (
    df['UV_1_280_corrected'] > 50)
df['High_Activity_2'] = (df['UV_2_260_corrected'] > df['UV_2_260_threshold']) | (
    df['UV_2_260_corrected'] > 50)

# Calculate Mean and StdDev of the calculated dynamic threshold per run for BOTH
    UV channels
# Vectorized groupby aggregation
threshold_stats_uv1 = df.groupby('run_no')['UV_1_280_threshold'].agg(['mean', 'std
    ']).to_dict('index')
threshold_stats_uv2 = df.groupby('run_no')['UV_2_260_threshold'].agg(['mean', 'std
    ']).to_dict('index')

# Combine into a single structure for logging
threshold_stats = {}
all_runs = set(threshold_stats_uv1.keys()) | set(threshold_stats_uv2.keys())
for run in all_runs:
    s1 = threshold_stats_uv1.get(run, {'mean': np.nan, 'std': np.nan})
    s2 = threshold_stats_uv2.get(run, {'mean': np.nan, 'std': np.nan})
    threshold_stats[run] = {
        'UV_1_280': {'mean': s1['mean'], 'std': s1['std']},
        'UV_2_260': {'mean': s2['mean'], 'std': s2['std']}
    }

# --- Output to Log ---

with open(OUTPUT_LOG, 'a') as f:
    f.write("## 1. Validation Results\n\n")
    f.write("### Mislabeled _ml Variables (per run)\n")
    for run, count in flagged_ml_counts.items():
        f.write(f"- Run {run}: {count} rows flagged\n")

    f.write("\n### Invalid Conductivity (per run)\n")
    for run, count in flagged_cond_counts.items():
        f.write(f"- Run {run}: {count} rows flagged\n")

    f.write("\n## 2. Baseline Strategy Summary\n\n")
    run_strategies = {}
    for key, strat in baseline_strategies.items():
        run = key.split('_')[0]
        if run not in run_strategies:
            run_strategies[run] = set()
        run_strategies[run].add(strat)

    for run, strats in run_strategies.items():
        f.write(f"- Run {run}: {', '.join(strats)}\n")

    f.write("\n## 3. Dynamic Threshold Statistics (per run)\n\n")
    for run, stats in threshold_stats.items():
        uv1 = stats['UV_1_280']
        uv2 = stats['UV_2_260']
        f.write(f"- Run {run}:\n")
        f.write(f"  - UV_1_280: Mean = {uv1['mean']:.4f}, StdDev = {uv1['std']:.4f
            }\n")
        f.write(f"  - UV_2_260: Mean = {uv2['mean']:.4f}, StdDev = {uv2['std']:.4f
            }\n")

print("Analysis complete. Log saved to /workspace/analysis_log.md")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_segmentation_comparative.png'
MAX_POINTS_PER_RUN = 1000

# Load Data
print(f"Loading data from {INPUT_FILE}...")

# Check file existence
if not os.path.exists(INPUT_FILE):
    raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")

# Read UV columns as object initially to handle non-numeric garbage safely
uv_cols_to_clean = ['UV_1_280_mAU', 'UV_2_260_mAU']
# Read the file first to check columns, then load with object dtype for UV cols
df_temp = pd.read_csv(INPUT_FILE, nrows=0)
uv_cols_present = [col for col in uv_cols_to_clean if col in df_temp.columns]
df = pd.read_csv(INPUT_FILE, dtype={col: object for col in uv_cols_present})

# Debug: Print available columns
print(f"Available columns in input file: {list(df.columns)}")

# Debug: Inspect raw UV data content
if 'UV_1_280_mAU' in df.columns:
    print("\nFirst 5 rows of UV_1_280_mAU (raw):")
    print(df['UV_1_280_mAU'].head())
    print("\nLast 5 rows of UV_1_280_mAU (raw):")
    print(df['UV_1_280_mAU'].tail())

if 'UV_2_260_mAU' in df.columns:
    print("\nFirst 5 rows of UV_2_260_mAU (raw):")
    print(df['UV_2_260_mAU'].head())
    print("\nLast 5 rows of UV_2_260_mAU (raw):")
    print(df['UV_2_260_mAU'].tail())

# Select relevant columns (core data)
cols_to_use = ['run_no', 'column', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Fraction_unique_ID',
               'Conc_B_%', 'chromatography_stage', 'volume_ml']

# Check if core required columns exist
missing_core_cols = [col for col in cols_to_use if col not in df.columns]
if missing_core_cols:
    raise ValueError(f"Missing core required columns in input file: {
        missing_core_cols}.")

df_analysis = df[cols_to_use].copy()

# Debug: Inspect Fraction_unique_ID types
print(f"\nUnique values of Fraction_unique_ID (first 10): {df_analysis['
```

```
    Fraction_unique_ID'].unique()[:10]}")
print(f"Type of Fraction_unique_ID: {df_analysis['Fraction_unique_ID'].dtype}")

# Explicit Cleaning for UV columns
for col in uv_cols_present:
    # Replace common non-numeric strings with NaN
    df_analysis[col] = df_analysis[col].replace(['N/A', '-', 'null', 'NULL', ''],
        np.nan)
    # Convert to numeric, coercing errors to NaN
    df_analysis[col] = pd.to_numeric(df_analysis[col], errors='coerce')

# Ensure numeric types for non-UV columns
# Note: Fraction_unique_ID is excluded from numeric coercion as per instructions
for col in ['Conc_B_%', 'volume_ml']:
    df_analysis[col] = pd.to_numeric(df_analysis[col], errors='coerce')

# Fallback Strategy: Handle missing 'threshold' and 'is_baseline_corrected'
# If 'threshold' is missing, calculate it dynamically per run
if 'threshold' not in df_analysis.columns:
    print("Warning: 'threshold' column missing. Calculating dynamic threshold (
        mean + 3*std) per run.")
    df_analysis['threshold'] = df_analysis.groupby('run_no')['UV_1_280_mAU'].
        transform(
        lambda x: x.mean() + 3 * x.std()
    )
else:
    df_analysis['threshold'] = pd.to_numeric(df_analysis['threshold'], errors='
        coerce')

# If 'is_baseline_corrected' is missing, create it and assume True to proceed
if 'is_baseline_corrected' not in df_analysis.columns:
    print("Warning: 'is_baseline_corrected' column missing. Assuming data is
        usable (True) to proceed.")
    df_analysis['is_baseline_corrected'] = True
else:
    # If it exists but is not boolean, try to convert or default to True
    if not df_analysis['is_baseline_corrected'].dtype == bool:
        print("Warning: 'is_baseline_corrected' is not boolean. Assuming True.")
        df_analysis['is_baseline_corrected'] = True

# 1. Segmentation & Threshold Validation
# Mark High Activity vs Baseline
# Ensure threshold is numeric for comparison
df_analysis['threshold'] = pd.to_numeric(df_analysis['threshold'], errors='coerce'
    )

df_analysis['segment_status'] = 'Baseline'
# Only mark as High Activity where both UV and threshold are valid numbers
valid_comparison = df_analysis['UV_1_280_mAU'].notna() & df_analysis['threshold'].
    notna()
df_analysis.loc[valid_comparison & (df_analysis['UV_1_280_mAU'] > df_analysis['
    threshold']), 'segment_status'] = 'High Activity'

# 2. Boundary Expansion (Fully Vectorized)
def expand_boundaries_vectorized(status_arr, ids_arr, volume_arr):
    n = len(status_arr)
    is_high = status_arr == 'High Activity'

    if not is_high.any():
```

```
        return is_high

    # Identify indices of High Activity
    high_indices = np.where(is_high)[0]

    if len(high_indices) < 2:
        return is_high

    # Calculate gaps between consecutive high segments
    gap_starts = high_indices[:-1] + 1
    gap_ends = high_indices[1:] - 1

    # Filter valid gaps (where start <= end)
    valid_mask = gap_starts <= gap_ends
    if not valid_mask.any():
        return is_high

    gap_starts = gap_starts[valid_mask]
    gap_ends = gap_ends[valid_mask]

    # Calculate gap properties
    gap_lengths = gap_ends - gap_starts + 1

    # Calculate gap duration using volume_ml
    gap_durations = volume_arr[gap_ends] - volume_arr[gap_starts]

    # Identify gaps to merge:
    # Condition 1: gap_length < 5 rows
    # Condition 2: gap_duration < 0.5 mins
    # Condition 3: gap consists of null Fraction_unique_ID AND gap_length <= 10

    # Vectorized check for Condition 1 & 2
    merge_mask = (gap_lengths < 5) | (gap_durations < 0.5)

    # Vectorized check for Condition 3 (Null IDs)
    # Updated to use pd.isna to handle object types (strings) or numeric NaNs
    is_null = pd.isna(ids_arr)

    # Use a cumulative sum of non-nulls to get O(1) range sum.
    non_null_count = np.cumsum(~is_null.astype(int))
    # Handle index 0
    non_null_count = np.insert(non_null_count, 0, 0)

    # Count non-nulls in gap [start, end] inclusive
    non_nulls_in_gap = non_null_count[gap_ends + 1] - non_null_count[gap_starts]

    all_null_mask = (non_nulls_in_gap == 0)

    # Combine Condition 3: All null AND length <= 10
    merge_mask |= (all_null_mask & (gap_lengths <= 10))

    # Apply merges using vectorized assignment
    if merge_mask.any():
        for i in np.where(merge_mask)[0]:
            start_idx = gap_starts[i]
            end_idx = gap_ends[i]
            is_high[start_idx:end_idx+1] = True

    return is_high
```

```
# Process per run
runs = df_analysis['run_no'].unique()
expanded_status_list = []

for run in runs:
    run_mask = df_analysis['run_no'] == run
    status_arr = df_analysis.loc[run_mask, 'segment_status'].to_numpy()
    ids_arr = df_analysis.loc[run_mask, 'Fraction_unique_ID'].to_numpy()
    volume_arr = df_analysis.loc[run_mask, 'volume_ml'].to_numpy()

    new_is_high = expand_boundaries_vectorized(status_arr, ids_arr, volume_arr)

    new_status = np.where(new_is_high, 'High Activity', 'Baseline')
    expanded_status_list.append(new_status)

expanded_status = np.concatenate(expanded_status_list)
df_analysis.loc[df_analysis['run_no'].isin(runs), 'segment_status'] =
    expanded_status

# 3. Contamination Flagging (Vectorized)
mask_high_activity = df_analysis['segment_status'] == 'High Activity'
mask_uv2_nan = df_analysis['UV_2_260_mAU'].isna()
mask_uv2_zero = df_analysis['UV_2_260_mAU'] == 0.0

# Debug: Inspect logical operation operands before line 166 equivalent
print("\n--- Debugging Logical Operation ---")
print(f"Type of mask_uv2_nan: {type(mask_uv2_nan)}, Value (first 5): {mask_uv2_nan
    .head()}")
print(f"Type of mask_uv2_zero: {type(mask_uv2_zero)}, Value (first 5): {
    mask_uv2_zero.head()}")

# Ensure operands are Series for bitwise OR
if isinstance(mask_uv2_nan, bool) or isinstance(mask_uv2_zero, bool):
    print("WARNING: One of the masks is a scalar boolean. This may cause logical
        errors.")

mask_uv2_invalid = mask_uv2_nan | mask_uv2_zero
print(f"Type of mask_uv2_invalid: {type(mask_uv2_invalid)}, Value (first 5): {
    mask_uv2_invalid.head()}")

mask_uv1_nan = df_analysis['UV_1_280_mAU'].isna()

# Debug: Check the comparison result
comparison_result = df_analysis['UV_2_260_mAU'] < 1.0
print(f"Type of (UV_2_260_mAU < 1.0): {type(comparison_result)}, Value (first 5): {comparison_result.head()}")

df_analysis['contamination_flag'] = 'None'

# Condition: High Activity AND (UV2 < 1.0 OR UV2 is NaN OR UV1 is NaN) -> 'Low
    Signal'
# Ensure all operands are Series
mask_low_signal = mask_high_activity & (comparison_result | mask_uv2_nan |
    mask_uv1_nan)
df_analysis.loc[mask_low_signal, 'contamination_flag'] = 'Low Signal'

# Condition: High Activity AND NOT Low Signal AND Ratio < 1.5 -> 'Potential
    Nucleic Acid Contamination'
```

```
valid_ratio_mask = mask_high_activity & ~mask_low_signal & ~mask_uv2_invalid & ~
    mask_uv1_nan

ratio = np.full(len(df_analysis), np.nan)
if valid_ratio_mask.any():
    ratio[valid_ratio_mask] = df_analysis.loc[valid_ratio_mask, 'UV_1_280_mAU'] /
        df_analysis.loc[valid_ratio_mask, 'UV_2_260_mAU']

mask_ratio_low = ratio < 1.5
mask_contam = mask_high_activity & ~mask_low_signal & mask_ratio_low
df_analysis.loc[mask_contam, 'contamination_flag'] = 'Potential␣Nucleic␣Acid␣
    Contamination'

# 4. High-Salt Wash Handling
df_analysis.loc[df_analysis['Conc_B_%'] > 80, 'segment_status'] = 'High-Salt␣Wash'

# Prepare for Visualization
def downsample_run(group):
    n = len(group)
    if n <= MAX_POINTS_PER_RUN:
        return group
    indices = np.linspace(0, n - 1, MAX_POINTS_PER_RUN, dtype=int)
    return group.iloc[indices]

df_plot = df_analysis.groupby('run_no', group_keys=False).apply(downsample_run)

# Create Multi-panel Figure
unique_runs = df_plot['run_no'].unique()
n_runs = len(unique_runs)
cols = 2
rows = (n_runs + cols - 1) // cols

fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows), sharex=False, sharey=
    False)
axes = axes.flatten()

color_baseline = 'lightgray'
color_peak = 'green'
color_threshold = 'orange'
color_contamination = 'red'
color_salt = 'purple'

for i, run in enumerate(unique_runs):
    ax = axes[i]
    run_data = df_plot[df_plot['run_no'] == run].sort_values('volume_ml')

    ax.plot(run_data['volume_ml'], run_data['UV_1_280_mAU'], label='UV␣280', color
        ='black', linewidth=0.5, alpha=0.7)

    # Only plot threshold if it exists and is valid
    if 'threshold' in run_data.columns and run_data['threshold'].notna().any():
        ax.plot(run_data['volume_ml'], run_data['threshold'], label='Threshold',
            color=color_threshold, linestyle='--', linewidth=1)

    mask_base = run_data['segment_status'] == 'Baseline'
    ax.fill_between(run_data['volume_ml'], run_data['UV_1_280_mAU'],
                    where=mask_base, color=color_baseline, alpha=0.3, label='
                        Baseline' if i==0 else "")
```

```
    mask_peak = run_data['segment_status'] == 'High Activity'
    ax.fill_between(run_data['volume_ml'], run_data['UV_1_280_mAU'],
                    where=mask_peak, color=color_peak, alpha=0.4, label='Peak' if
                        i==0 else "")

    mask_salt = run_data['segment_status'] == 'High-Salt Wash'
    ax.fill_between(run_data['volume_ml'], run_data['UV_1_280_mAU'],
                    where=mask_salt, color=color_salt, alpha=0.4, label='High-Salt
                        Wash' if i==0 else "")

    mask_contam = run_data['contamination_flag'] == 'Potential Nucleic Acid
        Contamination'
    if mask_contam.any():
        ax.scatter(run_data.loc[mask_contam, 'volume_ml'], run_data.loc[
            mask_contam, 'UV_1_280_mAU'],
                   color=color_contamination, marker='x', s=50, label='
                       Contamination' if i==0 else "")

    ax2 = ax.twinx()
    ax2.plot(run_data['volume_ml'], run_data['Conc_B_%'], color='blue', linestyle=
        ':', alpha=0.6, linewidth=1)
    ax2.set_ylabel('Conc_B_%', color='blue')
    ax2.tick_params(axis='y', labelcolor='blue')

    ax.set_title(f'Run {run}')
    ax.set_xlabel('Volume (ml)')
    ax.set_ylabel('UV 280 (mAU)')
    ax.grid(True, alpha=0.3)

for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle('Comparative Peak Segmentation and Contamination Analysis', fontsize
    =16, y=1.02)
plt.tight_layout(rect=[0, 0, 1, 0.98])

plt.savefig(OUTPUT_FILE, dpi=150, bbox_inches='tight')
print(f"Visualization saved to {OUTPUT_FILE}")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt

# File paths
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_LOG = '/workspace/analysis_log.md'
OUTPUT_DIR = '/workspace/'
BOUNDARY_FILE = '/inputs/expanded_boundaries.csv'

# Load data
df = pd.read_csv(INPUT_FILE)

# Ensure numeric types for calculations
numeric_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'Conc_B_%']
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
# Convert Fraction_unique_ID to numeric for integrity checks
df['Fraction_unique_ID'] = pd.to_numeric(df['Fraction_unique_ID'], errors='coerce'
    )

# Validation: Check for is_baseline_corrected column
# Fix: Create column with default True if missing to prevent immediate failure
if 'is_baseline_corrected' not in df.columns:
    df['is_baseline_corrected'] = True

if not df['is_baseline_corrected'].all():
    uncorrected_runs = df[~df['is_baseline_corrected']]['run_no'].unique()
    raise ValueError(f"Uncorrected␣baseline␣data␣detected␣in␣runs:␣{
        uncorrected_runs}.␣Analysis␣halted.")

# Step 1: Noise Floor Estimation
corrected_col = 'UV_1_280_mAU_corrected' if 'UV_1_280_mAU_corrected' in df.columns
    else 'UV_1_280_mAU'

mask_baseline = (df['chromatography_stage'] == 'Baseline') | (df['
    chromatography_stage'].isna())
mask_not_wash = df['Conc_B_%'] <= 80.0

baseline_df = df[mask_baseline & mask_not_wash]

noise_stats = baseline_df.groupby('run_no')[[corrected_col]].agg(['median', lambda
    x: x.quantile(0.95)])
noise_stats.columns = ['Noise_Median', 'Noise_95']
noise_stats = noise_stats.reset_index()

# Step 2: Peak Statistics
# Debug: Print unique values and counts to diagnose 'No peaks detected' error
print(f"Unique␣values␣in␣'chromatography_stage':␣{df['chromatography_stage'].
    unique()}")
print(f"Count␣of␣rows␣starting␣with␣'E'␣(Elution/Activity):␣{df['
    chromatography_stage'].str.startswith('E',␣na=False).sum()}")

peak_stage_mask = df['chromatography_stage'].str.startswith('E', na=False)

if not peak_stage_mask.any():
    raise ValueError("No␣peaks␣detected␣(stages␣starting␣with␣'E').␣Ensure␣'
        chromatography_stage'␣contains␣'E'␣values.")

peak_df = df[peak_stage_mask]

# Debug: Inspect Fraction_unique_ID in peak_df to diagnose missing IDs
print(f"Number␣of␣unique␣Fraction_unique_IDs␣in␣peak_df:␣{peak_df['
    Fraction_unique_ID'].nunique()}")
print(f"Count␣of␣NaN␣values␣in␣Fraction_unique_ID␣within␣peak_df:␣{peak_df['
    Fraction_unique_ID'].isna().sum()}")
non_nan_ids = peak_df['Fraction_unique_ID'].dropna().head(10).tolist()
nan_ids = peak_df[peak_df['Fraction_unique_ID'].isna()]['Fraction_unique_ID'].head
    (10).tolist()
print(f"First␣10␣non-NaN␣Fraction_unique_IDs␣in␣peak_df:␣{non_nan_ids}")
print(f"First␣10␣NaN␣Fraction_unique_IDs␣in␣peak_df␣(should␣be␣empty␣list␣if␣no␣
    NaNs):␣{nan_ids}")

# Calculate volume step (Vectorized)
df['volume_step'] = df.groupby('run_no')['volume_ml'].diff().fillna(0)
```

```
first_indices = df.groupby('run_no').head(1).index
df.loc[first_indices, 'volume_step'] = df.loc[first_indices, 'volume_ml']

# Determine grouping column: Use Fraction_unique_ID instead of peak_id
if 'Fraction_unique_ID' not in df.columns:
    raise ValueError("Missing required column 'Fraction_unique_ID'. Cannot group
        fractions into peaks.")
group_col = 'Fraction_unique_ID'

# Filter peak_df to only rows with valid group IDs
peak_df_valid = peak_df[peak_df[group_col].notna()]

# Fix: Do not raise error if peak_df_valid is empty. Log warning and proceed to
    anomaly detection.
if peak_df_valid.empty:
    print("Warning: No valid Fraction_unique_IDs found in peak regions. Proceeding
        to anomaly detection to flag missing IDs.")
    # Initialize empty peak_stats to handle downstream logic gracefully
    peak_stats = pd.DataFrame(columns=[group_col, 'run_no', 'Total_Peak_Volume', '
        Avg_Peak_Height', 'Max_Peak_Height', 'AUC_280', 'Width_Half_Height'])
else:
    # Pre-calculate product column for AUC
    peak_df_valid['signal_volume_product'] = peak_df_valid[corrected_col] *
        peak_df_valid['volume_step']

    # Vectorized AUC Calculation
    peak_stats = peak_df_valid.groupby(group_col).agg(
        run_no=('run_no', 'first'),
        Total_Peak_Volume=('volume_step', 'sum'),
        Avg_Peak_Height=(corrected_col, 'mean'),
        Max_Peak_Height=(corrected_col, 'max'),
        AUC_280=('signal_volume_product', 'sum')
    ).reset_index()

    # Vectorized Width Calculation (Feedback: Replace groupby().apply() with
        vectorized approach)
    peak_df_valid['group_max'] = peak_df_valid.groupby(group_col)[corrected_col].
        transform('max')
    peak_df_valid['half_height_threshold'] = peak_df_valid['group_max'] / 2
    peak_df_valid['is_above_half'] = peak_df_valid[corrected_col] >= peak_df_valid
        ['half_height_threshold']

    # Calculate width per group: sum of volume_step where is_above_half is True
    # Create a mask for the product of is_above_half and volume_step, then sum
    peak_df_valid['width_volume_step'] = peak_df_valid['volume_step'].where(
        peak_df_valid['is_above_half'], 0)
    width_stats = peak_df_valid.groupby(group_col)['width_volume_step'].sum().
        reset_index()
    width_stats.columns = [group_col, 'Width_Half_Height']

    peak_stats = peak_stats.merge(width_stats, on=group_col, how='left')
    peak_stats['Width_Half_Height'] = peak_stats['Width_Half_Height'].fillna(0)

# Step 3: Data Integrity Check (Expanded Boundary Limits)
# Fix: Handle missing boundary file gracefully instead of raising an error
if not os.path.exists(BOUNDARY_FILE):
    print(f"Warning: Expanded boundary data file not found at {BOUNDARY_FILE}.
        Skipping boundary check.")
    anomalies = pd.Series(dtype=float)
```

```
else:
    boundary_df = pd.read_csv(BOUNDARY_FILE)
    # Assume boundary_df has columns: run_no, min_ID, max_ID

    # Identify regions with missing Fraction_unique_ID in peak regions
    missing_id_mask = peak_stage_mask & df[group_col].isna()

    # Filter missing IDs against expanded boundaries
    # Logic: Only flag as anomalies if missing IDs fall *outside* the expanded
        boundaries.
    # Since ID is NaN, we check if volume_ml falls within min_ID and max_ID range
        (assuming ID correlates with volume or index).
    # If boundary_df uses volume ranges, we compare volume_ml. If it uses ID
        ranges, we assume min_ID/max_ID are volume thresholds.
    # Based on instructions: check if volume_ml is within min_ID and max_ID range
        for that run.

    missing_df = df[missing_id_mask].copy()
    anomalies = pd.Series(dtype=float)

    if 'run_no' in missing_df.columns and 'run_no' in boundary_df.columns:
        # Merge to get boundaries for each missing row
        missing_with_bounds = missing_df.merge(boundary_df, on='run_no', how='left
            ')

        # Check if volume_ml is within [min_ID, max_ID]
        # Assuming min_ID and max_ID in boundary_df represent the valid volume
            range for the peak collection
        valid_mask = (
            missing_with_bounds['volume_ml'].notna() &
            missing_with_bounds['min_ID'].notna() &
            missing_with_bounds['max_ID'].notna() &
            (missing_with_bounds['volume_ml'] >= missing_with_bounds['min_ID']) &
            (missing_with_bounds['volume_ml'] <= missing_with_bounds['max_ID'])
        )

        # Anomalies are rows where volume is OUTSIDE the range (or bounds missing)
        anomaly_mask = ~valid_mask
        anomaly_rows = missing_with_bounds[anomaly_mask]

        if not anomaly_rows.empty:
            # Calculate anomaly percentage based on volume
            total_ha_volume_per_run = df[peak_stage_mask].groupby('run_no')['
                volume_step'].sum()
            anomaly_vol_per_run = anomaly_rows.groupby('run_no')['volume_step'].
                sum()

            # Ensure alignment and avoid division by zero
            anomaly_pct_per_run = (anomaly_vol_per_run / total_ha_volume_per_run)
                * 100
            anomaly_pct_per_run = anomaly_pct_per_run.fillna(0)

            # Flag runs where anomaly_pct > 5
            anomalies = anomaly_pct_per_run[anomaly_pct_per_run > 5].sort_values(
                ascending=False).head(5)
        else:
            anomalies = pd.Series(dtype=float)
    else:
        anomalies = pd.Series(dtype=float)
```

```
# Step 4: Summary Generation
# 1. Total Peaks
if not peak_stats.empty:
    peak_counts = peak_stats.groupby('run_no')[group_col].nunique().reset_index()
    peak_counts.columns = ['run_no', 'Total_Peaks']
else:
    peak_counts = pd.DataFrame(columns=['run_no', 'Total_Peaks'])

# 2. Avg Peak Height
if not peak_stats.empty:
    avg_peak_height_per_run = peak_stats.groupby('run_no')['Avg_Peak_Height'].mean
        ().reset_index()
    avg_peak_height_per_run.columns = ['run_no', 'Avg_Peak_Height']
else:
    avg_peak_height_per_run = pd.DataFrame(columns=['run_no', 'Avg_Peak_Height'])

# 3. Noise Floor 95th
noise_95_per_run = noise_stats[['run_no', 'Noise_95']]

# 4. Contamination Rate
if 'is_contaminated' in df.columns:
    contamination_mask = df['is_contaminated']
else:
    contamination_mask = df['chromatography_stage'] == 'Contamination'

df['is_contaminated_flag'] = contamination_mask
contamination_counts = df.groupby('run_no')['is_contaminated_flag'].sum()
total_counts = df.groupby('run_no').size()
contamination_rate = (contamination_counts / total_counts * 100).reset_index()
contamination_rate.columns = ['run_no', 'Contamination_Rate_%']

# 5. High Salt Wash Duration
wash_mask = (df['Conc_B_%'] > 80) | (df['chromatography_stage'] == 'High␣Salt␣Wash
    ')
wash_duration = df[wash_mask].groupby('run_no')['volume_step'].sum().reset_index()
wash_duration.columns = ['run_no', 'High_Salt_Wash_Duration_min']

# Merge summaries
summary_df = pd.DataFrame({'run_no': df['run_no'].unique()})
summary_df = summary_df.merge(peak_counts, on='run_no', how='left')
summary_df = summary_df.merge(avg_peak_height_per_run, on='run_no', how='left')
summary_df = summary_df.merge(noise_95_per_run, on='run_no', how='left')
summary_df = summary_df.merge(contamination_rate, on='run_no', how='left')
summary_df = summary_df.merge(wash_duration, on='run_no', how='left')

# Fill NaNs
summary_df['Total_Peaks'] = summary_df['Total_Peaks'].fillna(0)
summary_df['Avg_Peak_Height'] = summary_df['Avg_Peak_Height'].fillna(0)
summary_df['Noise_95'] = summary_df['Noise_95'].fillna(0)
summary_df['Contamination_Rate_%'] = summary_df['Contamination_Rate_%'].fillna(0)
summary_df['High_Salt_Wash_Duration_min'] = summary_df['
    High_Salt_Wash_Duration_min'].fillna(0)

# Noise Floor Stats
noise_floor_mean = summary_df['Noise_95'].mean()
noise_floor_std = summary_df['Noise_95'].std()

# Generate Charts
```

```
plt.figure(figsize=(10, 6))
plt.plot(summary_df['run_no'], summary_df['Noise_95'], marker='o')
plt.title('Noise Floor (95th Percentile) vs Run')
plt.xlabel('Run No')
plt.ylabel('Noise Floor (mAU)')
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'noise_floor_vs_run.png'))
plt.close()

plt.figure(figsize=(10, 6))
plt.hist(summary_df['Avg_Peak_Height'], bins=20, edgecolor='black')
plt.title('Distribution of Average Peak Height')
plt.xlabel('Average Peak Height (mAU)')
plt.ylabel('Frequency')
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'peak_height_distribution.png'))
plt.close()

# Generate Output Log
with open(OUTPUT_LOG, 'a') as f:
    f.write("\n--- Run-Level Noise Floor Estimation and Quality Control Summary (
        Step 3) ---\n")

    f.write("\n**Run Summary Table**\n")
    f.write("Columns: run_no, Total_Peaks, Avg_Peak_Height, Noise_Floor_95th, 
        Contamination_Rate_%, High_Salt_Wash_Duration_min\n")
    f.write("run_no,Total_Peaks,Avg_Peak_Height,Noise_Floor_95th,
        Contamination_Rate_%,High_Salt_Wash_Duration_min\n")

    # Vectorized string formatting for log (Feedback: Replace iterrows with to_csv
        )
    head_df = summary_df.head(15).copy()
    head_df['Avg_Peak_Height'] = head_df['Avg_Peak_Height'].apply(lambda x: f"{x
        :.2f}")
    head_df['Noise_95'] = head_df['Noise_95'].apply(lambda x: f"{x:.2f}")
    head_df['Contamination_Rate_%'] = head_df['Contamination_Rate_%'].apply(lambda
         x: f"{x:.2f}")
    head_df['High_Salt_Wash_Duration_min'] = head_df['High_Salt_Wash_Duration_min'
        ].apply(lambda x: f"{x:.2f}")

    f.write(head_df.to_csv(index=False, header=False))

    if len(summary_df) > 15:
        f.write(f"... (Total {len(summary_df)} runs, showing first 15)\n")

    # Anomaly Report (Feedback: Replace iterative loop with vectorized string
        formatting)
    f.write("\n**Anomaly Report** (>5% High Activity outside collection windows)\n
        ")
    if len(anomalies) > 0:
        anomaly_report = anomalies.apply(lambda x: f"Run {x.name}: {x:.2f}% 
            anomaly")
        f.write(anomaly_report.to_string(index=False))
        f.write("\n")
    else:
        f.write("No anomalies detected (>5% threshold).\n")
```

```
    # Noise Floor Stats
    f.write(f"\n**Noise Floor Stats**\n")
    f.write(f"Mean Noise Floor (95th percentile): {noise_floor_mean:.2f}\n")
    f.write(f"StdDev Noise Floor (95th percentile): {noise_floor_std:.2f}\n")

    # Chart references
    f.write(f"\n**Generated Charts**\n")
    f.write(f"- Noise Floor vs Run: {os.path.join(OUTPUT_DIR, 'noise_floor_vs_run.
        png')}\n")
    f.write(f"- Peak Height Distribution: {os.path.join(OUTPUT_DIR, '
        peak_height_distribution.png')}\n")

print("Step 3 completed. Results appended to analysis_log.md")
```

Listing 3: Code_3